UNPAIRED IMAGE-TO-IMAGE TRANSLATION WITH A HAND-CODED CYCLEGAN IN NUMPY

Baptiste Pras
M1 Artificial Intelligence
baptiste.pras@universite-paris-saclay.fr
github.com/baptistepras/CycleGAN

ABSTRACT

In this work we re-implement CycleGAN, a model for unpaired image-to-image translation, using only NumPy. We do not use PyTorch, TensorFlow, or any autograd library. Every forward and backward pass is written by hand. We follow the architecture and the losses of the original paper: a ResNet generator, a PatchGAN discriminator, the LSGAN adversarial loss, a cycle consistency loss and an identity loss. Because we run on CPU and we have no autograd, we lower the image size and we use a subset of the horse2zebra and apple2orange datasets. The goal of this report is to understand each piece of CycleGAN by writing it ourselves, and to see if we can still observe a cycle consistency effect on a small setup.

1 INTRODUCTION

Image-to-image translation is the task of changing an image from one visual style (one "domain") into another. For example, turning a horse into a zebra, a photo into a painting, or a summer landscape into a winter one. A common way to do this is to train a model on paired images: the same scene shown in domain X and in domain Y. But getting paired data is often hard, expensive, or simply impossible. There is no real "horse version" and "zebra version" of the same photo.

CycleGAN (Zhu et al., 2017) was made exactly for this case: learning a translation between two domains when we only have two unpaired sets of images. The main idea is the cycle consistency. If we translate a horse into a zebra, and then translate this zebra back, we should recover the original horse. This is a strong constraint and it allows the model to learn without paired supervision. In practice the model is composed of two generators (one for each direction) and two discriminators (one for each domain).

In this miniproject we re-implement CycleGAN by hand, in pure NumPy. We do not use PyTorch or autograd. The idea is to really understand what is inside the model: how a convolution is computed, how its gradient flows, and how the two generators and two discriminators are trained together. A pure NumPy implementation is much slower than a GPU framework, so we work on smaller images and a subset of the dataset, but we keep the structure of the model close to the original.

The research question of this report is: can we re-implement CycleGAN by hand in NumPy, and still observe the cycle consistency effect, on a small compute budget?

2 RELATED WORKS

2.1 GENERATIVE ADVERSARIAL NETWORKS

GANs were introduced by Goodfellow et al. (2014). Two networks are trained at the same time: a generator that produces fake samples, and a discriminator that tries to tell real samples from fake ones. They play a min-max game, and at the end the generator should produce samples that look real. The original loss is a cross-entropy. Later, Mao et al. (2017) proposed LSGAN, which uses a least-squares loss instead. LSGAN is known to be more stable and to give cleaner gradients. CycleGAN uses LSGAN, and we also do.

2.2 PAIRED IMAGE-TO-IMAGE TRANSLATION

Isola et al. (2017) (pix2pix) train a conditional GAN on pairs of images. The generator gets an input image and produces an output image in another domain. The discriminator looks at small patches of the image (PatchGAN) and judges if each patch looks real. This gives a more local supervision and works well in practice. The strong limit of pix2pix is that it needs paired training data.

2.3 UNPAIRED IMAGE-TO-IMAGE TRANSLATION

CycleGAN (Zhu et al., 2017) removes the paired-data assumption thanks to the cycle consistency. At the same period, two other works proposed a very similar idea: DiscoGAN (Kim et al., 2017) and DualGAN (Yi et al., 2017). The three works are different in details but share the same core message: with two generators and a cycle constraint, you can learn to translate without pairs. Before these GAN-based methods, neural style transfer (Gatys et al., 2016) was already able to change the style of an image, but it worked one image at a time and used a pretrained CNN.

2.4 ARCHITECTURE AND TRAINING TRICKS

The generator we use comes from Johnson et al. (2016), who designed a small ResNet for fast neural style transfer. Instance normalization (Ulyanov et al., 2016) normalizes per image and per channel, instead of per batch. It works better than batch norm for style tasks, and CycleGAN uses it. To stabilize the discriminator, Shrivastava et al. (2017) introduced an *image buffer*: instead of always seeing the latest fake images, the discriminator also sees a small history of past fakes. This avoids the model and the discriminator chasing each other on a single image. Finally, the optimizer is Adam (Kingma and Ba, 2015), which is also what we use.

2.5 HAND-CODED DEEP LEARNING

Our re-implementation is in NumPy only. The convolution is done with `im2col` and `col2im`.

3 METHOD

3.1 SETTING AND NOTATION

We have two image domains X and Y, with samples $x \sim p_X$ and $y \sim p_Y$. The two sets are *not* paired: we do not know which x corresponds to which y. We want to learn two mappings:

$$G : X \to Y, \qquad F : Y \to X. \tag{1}$$

We also train two discriminators: D_Y tries to tell real y from $G(x)$, and D_X tries to tell real x from $F(y)$.

3.2 LOSSES

Adversarial loss (LSGAN form). For G and D_Y:

$$\mathcal{L}_{D_Y} = \tfrac{1}{2}\mathbb{E}_y\big[(D_Y(y) - 1)^2\big] + \tfrac{1}{2}\mathbb{E}_x\big[D_Y(G(x))^2\big], \tag{2}$$

$$\mathcal{L}_{G,Y} = \mathbb{E}_x\big[(D_Y(G(x)) - 1)^2\big]. \tag{3}$$

Symmetric formulas hold for F and D_X.

Cycle consistency. If we send x to domain Y and back, we should recover x:

$$\mathcal{L}_{cyc} = \mathbb{E}_x\big[\|F(G(x)) - x\|_1\big] + \mathbb{E}_y\big[\|G(F(y)) - y\|_1\big]. \tag{4}$$

We use ℓ_1 because it gives sharper images than ℓ_2 (this is also reported in pix2pix).

Identity loss. If we feed G an image already in Y, it should not change much:

$$\mathcal{L}_{id} = \mathbb{E}_y\big[\|G(y) - y\|_1\big] + \mathbb{E}_x\big[\|F(x) - x\|_1\big]. \tag{5}$$

This helps the generators to keep the colors of the input stable, especially for tasks like photo $\leftrightarrow$ painting. It is optional, but the paper recommends it.

Total objective. The full loss is

$$\mathcal{L} = \mathcal{L}_{GAN}(G, D_Y) + \mathcal{L}_{GAN}(F, D_X) + \lambda_{cyc}\mathcal{L}_{cyc} + \lambda_{id}\lambda_{cyc}\mathcal{L}_{id}. \tag{6}$$

We minimize $\mathcal{L}$ with respect to G and F, and maximize it with respect to D_X and D_Y.

3.3 Architecture

Generator. We use the ResNet generator of Johnson et al. (2016). The structure is:

- Initial block: `ReflectionPad(3) - Conv(7,1,64) - InstanceNorm - ReLU`.
- Two downsampling blocks: `Conv(3,2,c) - InstanceNorm - ReLU`, with c going $64 \to 128 \to 256$.
- n_{res} residual blocks at 256 channels (the paper uses 6 or 9; we use fewer).
- Two upsampling blocks: `NearestUpsample(2) - Conv(3,1,c) - InstanceNorm - ReLU`, with c going $256 \to 128 \to 64$.
- Final block: `ReflectionPad(3) - Conv(7,1,3) - Tanh`.

We replaced the transposed convolutions of the paper by nearest-neighbor upsampling followed by a normal convolution. This is to avoid checkerboard artifacts (Odena et al., 2016), and it is also much easier to backpropagate by hand.

Discriminator. We use the 70×70 PatchGAN from Isola et al. (2017). It is a small CNN that outputs a 2D map of scores. Each value of the map judges if a patch of the input image looks real or fake. The structure is: `Conv(4,2,64) - LeakyReLU(0.2)`, then three blocks of `Conv(4,2,c) - InstanceNorm - LeakyReLU(0.2)` with c going $64 \to 128 \to 256 \to 512$, and a final `Conv(4,1,1)`.

3.4 Hand-coded NumPy implementation

Every layer is a small Python class with two methods: a `forward` that stores what it needs in a cache, and a `backward` that uses the cache to return the gradient with respect to its inputs and to its parameters. There is no autograd. Convolutions use `im2col` and `col2im`. Adam is also coded by hand, with the bias correction. The image buffer of Shrivastava et al. (2017), with size 50, is used to feed the discriminators.

One important detail is the order of the calls. In one training step, G and F are called several times (cycle $A \to B \to A$, cycle $B \to A \to B$, identity), and each call overwrites the cache of the layers. So we do the full `forward + backward` of one path before starting the next, otherwise the cached values would not match and the gradients would be wrong.

4 Experimental Setup

4.1 Dataset

We use two datasets from the original CycleGAN paper, both downloaded from `https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/`. `horse2zebra` contains around 1000 horse and 1300 zebra images; it is a texture translation task (drawing stripes). `apple2orange` contains around 1000 images per side; it is a simpler color translation task (changing hue from red/green to orange). Both sets are completely unpaired.

Because our implementation is in pure NumPy, on CPU only, and without autograd, we cannot afford the original training setup (256×256 images, full dataset, 200 epochs). We made two compromises, that we report and discuss:

- We resize the images to a smaller resolution (64×64 instead of 256×256).
- We use only a subset of each domain (we cap the number of images per side with a `--max_per_side` flag).

4.2 MODEL PARAMETERS

We follow the original paper for the structure of G, F, D_X, D_Y but we reduce the depth of the model so that it can fit on CPU. The configuration we report on is:

- Generator base width `ngf` = 64 (same as the paper).
- Number of residual blocks $n_{res} = 6$ (the paper uses 6 for 128×128 and 9 for 256×256).
- Discriminator base width `ndf` = 64 (same as the paper).
- Image size: 64×64.
- Input/output channels: 3 (RGB).

Weights are initialized with a Gaussian of mean 0 and standard deviation 0.02, like in the paper.

4.3 TRAINING PARAMETERS

We keep the hyperparameters of the paper but with a smaller scale:

- Optimizer: Adam, $\beta_1 = 0.5$, $\beta_2 = 0.999$, learning rate 2×10^{-4}.
- Batch size: 1 (this is also what the original paper uses).
- Number of epochs: 20 (10 with constant learning rate, then 10 with linear decay to zero), instead of 200 in the paper. We use 150 images per side (out of ≈1000 available).
- Cycle weight $\lambda_{cyc} = 10$, identity weight $\lambda_{id} = 0.5$ (so the identity term has weight $0.5 \times 10 = 5$ in the total loss).
- Image buffer size: 50.

The discriminator loss is divided by 2, as in the paper. We implement this by scaling the gradients by 0.5 before `opt_D.step()`, instead of dividing the loss before the backward.

4.4 EVALUATION METRICS

The original CycleGAN paper uses several evaluation methods:

- AMT human perceptual studies (real vs fake) for tasks like aerial photos.
- FCN-score on Cityscapes, which uses a pretrained semantic segmentation network on the target domain.
- Standard segmentation metrics (per-class accuracy and IoU) on Cityscapes.

None of these are easy to reproduce in our setting. AMT studies need human raters, and FCN-score requires a pretrained segmentation network on the target domain (Cityscapes), which we do not have for horse / zebra. Training such a network ourselves is out of scope.

We therefore use simpler metrics, which we think are still informative on a small setup:

- **Qualitative samples.** Every `sample_every` steps we save a grid with $(a, G(a), F(G(a)))$ on one row and $(b, F(b), G(F(b)))$ on another row. The visual check is the main signal: does the model produce something that looks like a zebra (or a horse)? Does the cycle reconstruction recover the input?
- **Cycle reconstruction error.** The mean ℓ_1 distance between x and $F(G(x))$ on the test set, and similarly for the other direction. A small value means that the two generators are close to inverses of each other. It is not enough alone to prove that the translation is good, but it is necessary.
- **Loss curves.** We log the five losses (G adversarial, cycle, identity, D_X, D_Y) at every step. We expect the GAN losses to oscillate around a small value (this is the GAN equilibrium), and the cycle loss to clearly decrease over training.

Table 1: Training losses (running average over epoch) at five checkpoints. "cyc" and "id" are sums over both directions.

horse2zebra

Ep	g	**cyc**	**id**	d_A	d_B
1	1.215	0.726	0.690	1.106	1.182
5	0.648	0.535	0.498	0.253	0.233
10	0.715	0.477	0.441	0.194	0.161
15	0.805	0.400	0.376	0.081	0.063
20	0.755	0.316	0.310	0.039	0.039

apple2orange

Ep	g	**cyc**	**id**	d_A	d_B
1	1.338	0.842	0.768	1.125	1.237
5	0.620	0.572	0.516	0.274	0.239
10	0.727	0.510	0.461	0.186	0.199
15	0.779	0.408	0.378	0.096	0.077
20	0.777	0.306	0.299	0.039	0.036

Table 2: Test-set ℓ_1 metrics at epoch 20 on both datasets. Lower is better.

Metric	**horse2zebra**	**apple2orange**
$\|F(G(x)) - x\|_1$ $(A \to B \to A)$	0.1991	0.2017
$\|G(F(y)) - y\|_1$ $(B \to A \to B)$	0.2236	0.2192
$\|F(x) - x\|_1$ (identity on X)	0.1839	0.1889
$\|G(y) - y\|_1$ (identity on Y)	0.2162	0.2128

4.5 Hardware and runtime

All experiments run on a MacBook with Apple Silicon, on CPU only. One training step takes about 2.5 s at 64×64, so a full run of 20 epochs over 150 images per side takes around 2.5 hours. The runtime is the main reason we lowered the image size and capped the dataset size.

5 Results and Discussion

We trained the model on both datasets with the same configuration: 150 images per side, 64×64 resolution, $\texttt{ngf} = \texttt{ndf} = 64$, $n_{res} = 6$, batch size 1, 20 epochs (10 constant lr, 10 linear decay). Loss curves for both datasets are reported in Table 1.

5.1 Training dynamics

Table 1 reports the five losses at five checkpoints for both datasets. The dynamics are very consistent: the cycle loss decreases clearly on both (horse2zebra: $0.726 \to 0.316$; apple2orange: $0.842 \to 0.306$), confirming that the cycle constraint is learned regardless of the domain pair. The identity loss follows the same trend, and the generator GAN loss g stabilizes around 0.7–0.8 after the first epoch.

The discriminator losses are more concerning. They start at ≈ 1.1–1.2, cross the LSGAN equilibrium of 0.25[1] around epoch 5–6 on both datasets, and keep dropping to ≈ 0.04 at epoch 20. D dominates, and we discuss this below.

5.2 Test-set metrics

We evaluate cycle consistency on the test splits (never seen during training). Results are in Table 2. The cycle ℓ_1 is around 0.20–0.22 on both datasets, about 10% pixel error on $[-1, 1]$. The two datasets give nearly identical numbers, which shows that the cycle constraint generalizes across translation tasks and is not specific to one domain pair. A generalization gap remains (training cyc ≈ 0.31, test cyc ≈ 0.42), expected with 150 images per side.

5.3 Qualitative samples

Each output grid shows $(x, \hat{y}, \hat{x})$ where $\hat{y} = F(y)$ is the translated image and $\hat{x} = G(F(y))$ is the reconstruction. Figure 1 shows one example per dataset. On `horse2zebra` ($B \to A$) the model

[1] If D is fooled, $D(\text{real}) \approx D(\text{fake}) \approx 0.5$ and the LSGAN D loss equals $\frac{1}{2}(0.5^2 + 0.5^2) = 0.25$.

removes the stripes and produces a brownish coat; on `apple2orange` ($B \to A$) the color shift from orange to red/green is very clean. The $A \to B$ direction (adding stripes or changing to orange) is harder and less convincing. In both cases, the reconstruction recovers the input well.

(a) `horse2zebra`: zebra → horse → rec.

(b) `apple2orange`: orange → apple → rec.

Figure 1: $B \to A$ translations (64×64). Each strip: input → translated → cycle reconstruction.

5.4 Discussion

The two main observations from these results are:

Cycle consistency works, and it generalizes across tasks. The cycle loss decreases monotonically on both datasets, with nearly identical test metrics (≈ 0.20). This confirms that the cycle constraint is not specific to one domain pair: it is a general learning signal. The visual quality is notably better on `apple2orange` because changing hue is easier than inventing stripes, the generator only needs to shift colors, not create new spatial structure.

The discriminator becomes too strong in the second half of training. The d_A and d_B values drop below the LSGAN equilibrium of 0.25 already around epoch 5, and reach 0.039 at epoch 20. When this happens, the GAN gradients become weaker and the generator stops improving visually. With only 150 images per side, D has an easy job: it can almost memorize the real patches. A common fix is to regularize D (label smoothing, R_1 gradient penalty, or more data); we did not implement these, in order to stay close to the original paper.

A second limit is the resolution. At 64×64, the overall shape and color of the animal change, but fine structures like zebra stripes are still blurry. The original paper uses 256×256 images, which we could not afford on CPU.

6 Conclusion

We re-implemented CycleGAN by hand in pure NumPy, with no autograd and no PyTorch, following the original architecture and losses. On 150 images per side of `horse2zebra` at 64×64, the cycle loss decreases monotonically and the test-set cycle reconstruction reaches about 0.20 on a $[-1, 1]$ pixel range, so the central idea of the paper is reproducible even with limited compute.

The main limitations are: (i) resolution still well below the paper's 256×256; (ii) small dataset, which lets D dominate; (iii) only 20 epochs; (iv) no perceptual / FCN-based metric. Possible improvements: regularize D, use more data, train longer, and add a perceptual metric. A natural follow-up would be to write a small autograd engine on top of our `Module` class, so that future projects can reuse the same NumPy backbone without manual backward passes.

References

Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. Image style transfer using convolutional neural networks. In *CVPR*, 2016.

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *NeurIPS*, 2014.

Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *CVPR*, 2017.

Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *ECCV*, 2016.

Taeksoo Kim, Moonsu Cha, Hyunsoo Kim, Jung Kwon Lee, and Jiwon Kim. Learning to discover cross-domain relations with generative adversarial networks. In *ICML*, 2017.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

Xudong Mao, Qing Li, Haoran Xie, Raymond Y.K. Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks. In *ICCV*, 2017.

Augustus Odena, Vincent Dumoulin, and Chris Olah. Deconvolution and checkerboard artifacts. *Distill*, 2016.

Ashish Shrivastava, Tomas Pfister, Oncel Tuzel, Joshua Susskind, Wenda Wang, and Russell Webb. Learning from simulated and unsupervised images through adversarial training. In *CVPR*, 2017.

Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv:1607.08022*, 2016.

Zili Yi, Hao Zhang, Ping Tan, and Minglun Gong. DualGAN: Unsupervised dual learning for image-to-image translation. In *ICCV*, 2017.

Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *ICCV*, 2017.